Online Resource 2: Supplementary Addendum

The Review–Execution Gap in Agentic Software: Approval Scope and Recovery

Liang Hanghao, Xuan Wentao, Chen Qile, and Peng Peng

Automated Software Engineering manuscript
College of Computer Science and Electronic Engineering, Hunan University, Changsha, China
Corresponding author: Peng Peng, hnu16pp@hnu.edu.cn

Scope. This resource accompanies the manuscript on the review–execution gap. Online Resource 1 retains the original study documentation. The original 512-arm collection and separately frozen 117-pair deployment-A collection keep their original denominators, scores and inference. The analyses below are later diagnostics or separately frozen exploratory studies. Deterministic implementation checks are distinguished from hosted-agent observations. Sections A1–A2 report output-budget evidence, A3 examines receipt semantics, A4 reports local mechanism probes, A5 documents the historical audit, A6 maps controlled cases, A7 recounts preview exposure, and A8 reports configured ToolGate and public-reference workflows. Sections A9–A10 give the extended specification and design-selection guide; A11 documents native-approval and binding-scope reproduction; A12 reconstructs recovery availability and matched routes; A13 reports a separately frozen permission-state and feedback comparison; A14 tests native-to-store composition; A15 retains the detailed obligation-to-foundation map; A16 reports a separately frozen dynamic-recovery pilot; A17 reports a separately frozen check of reviewed-argument delivery across a process boundary in three pinned SDKs.

A1 Output truncation in original deployment B

The diagnostic audit covered all 128 B tasks and 256 arms, counting each shared prefix once. It linked all 1,372 recorded provider responses to their events and saved inputs. Of these responses, 1,293 ended naturally with parseable actions and 79 reported `stop_reason=max_tokens`, with no parseable action. All 79 occurred after the shared review prefix: 76 had no text and three contained partial, unparseable text. The 4,096-token cap is established by the frozen configuration and adapter, not a retained HTTP wire capture.

Table 1: Original B diagnostic counts. The original outcomes are retained.

Quantity	Context	Bound
Arms	128	128
Arms with at least one truncated suffix response	33	28
Truncated suffix responses	43	36
Original invalid-output terminations	10	8
Original task completions	117	119

All eighteen invalid-output terminations ended in a truncated response after the history's one format-repair allowance had already been used. Five histories contained intervening parseable actions between their two truncated responses; these were not uniformly consecutive failures or transport retries. Among all 61 affected arms, 42 later completed, eighteen ended with invalid output, and one retained its missing-termination classification. No transport retry was recorded. Truncation at any point therefore cannot be equated with task failure.

The original bound-minus-context completion difference of two tasks coincides arithmetically with the unequal locations of these terminations: seven pairs have only a context termination, five only a bound termination, and three both. This is a descriptive decomposition, not an estimate of what the historical model would have done with a larger budget. The original failure scores, unknown handling and integrity violations remain intact; a later reporting failure does not undo a committed instance violation. The diagnostic supports describing these outcomes as truncation-associated invalid output, rather than naturally completed format noncompliance.

A2 Separately frozen output-budget sensitivity study

Twenty-four tasks were sampled from the original task definitions, four from each of E, G, L, N, R and V. Each archived B review checkpoint and its visible prefix was continued under a contemporaneous 2×2 design: context/bound policy crossed with caps of 4,096/393,216 output tokens. The resulting 96 continuations were assigned before collection, with task blocks and Williams arm order fixed. There was one continuation per task and condition. This equal-scenario sample is not weighted like the original 128-task set.

Both budgets used the same current deployment, temperature zero, and extended waiting limits of 1,800 seconds per request and 21,600 seconds per continuation. The original twenty decision steps, twenty-four tool calls and one format-repair allowance per history were retained. The extended waiting limits also apply to the lower-cap condition; neither condition is an exact rerun of the original 90/720-second limits. The model alias does not establish that the immutable model version matched the historical collection.

Table 2: Complete 96-continuation sensitivity results. U and I are scored separately; there are no unknown outcomes in this collection.

Policy	Output cap	$U = 1$	$I = 1$	Invalid output	Truncated/responses
Context	4,096	23/24	20/24	1	6/100
Bound	4,096	22/24	24/24	2	8/93
Context	393,216	24/24	20/24	0	0/96
Bound	393,216	24/24	24/24	0	0/100

The lower-cap conditions produced fourteen truncated responses across eleven arms. Three arms terminated with invalid output after truncation, before any tool action; they remain in the denominator and cost totals. In the expanded-cap conditions, all 196 responses ended naturally; thirteen reported more than 4,096 output tokens, with maximum 8,447. Thus, the larger cap was used in practice. All 389 requests received responses, with no outer transport retry.

The descriptive bound-minus-context completion difference is $-1/24$ at the lower cap and zero at the expanded cap. The difference of these differences is $+1/24$ (4.17 percentage points), supported by only three discordant tasks. No equivalence, non-inferiority or stable interaction claim follows. The four context integrity violations at each cap occurred at the programmed G handoff, not through naturally occurring agent substitution. Bound recovered with reviewed-candidate reuse in all four G tasks under each budget; those are two observations of the same four tasks, not eight independent tasks.

These observations show budget sensitivity in the sampled current continuations while retaining the integrity separation under both caps. They neither replace original results nor recover the counterfactual outcome of the eighteen historical failures. Actual continuation time and provider usage are available in the artifact report; low tool counts in prematurely terminated arms are not evidence of efficiency.

A3 Receipt unavailability versus uncertain commit outcome

In the original R family, the agent receives a successful commit response before the first subsequent receipt query becomes unavailable. Authoritative state remains readable. A state-confirmation endpoint in that setting concerns an already acknowledged commit; it does not show that state inspection alone can attribute a commit whose success response was never delivered.

A separate deterministic check used the unchanged original Host, with durable copies of the full original request and key. Accept ($20 \to 20$) and Correction ($20 \to 21$), two policies and six history conditions produced 24 main records, plus eight controls. The fixed recovery script retried the original arguments and key once and then read full public state. It received only the client journal and public returns; injected history and private state were used only by the environment and scorer.

There were twelve confirmed, four rejected and eight unknown reports, with one business transition in every final state. Four paired hidden histories had identical full client journals and public observations, including values, versions, sources and capabilities, but different request ownership. The competing request used the same candidate and grant with another key; it was not an independently approved actor. These pairs are concrete witnesses that a matching state need not identify the request that committed. They do not prove that all possible interfaces are permanently insufficient.

Controls also distinguished historical receipts from current permission: the original successful response remained replayable after reviewer disablement without a new transition, and a cached refusal remained replayable after reviewer restoration while a valid fresh request could succeed. Changed arguments under a

Table 3: Deterministic recovery witnesses. Each row contains four records. These are not LLM-agent outcomes.

History	Correct own-request report	New recovery transitions
Initial request not delivered	Confirmed after retry	One per record
Commit persisted, success reply lost	Historical success	Zero
Competing key commits same value	Rejected; goal true	Zero
Own commit, then reviewer disabled	Historical success	Zero
Own commit; retry delivery blocked	Unknown; goal true	Zero
Competing commit; retry delivery blocked	Unknown; goal true	Zero

previously used key were rejected as a collision; such a collision is not evidence that the original request failed.

Both runs passed all 316 implementation checks; rebuilding in another directory was not a new statistical sample. The result depends on a complete retained client request, retained idempotency records and complete SQLite state checkpoints. It does not cover partial persistence, process crashes during commit, cache eviction, lost client journals, real concurrent races or cross-service effects. Existing retry and receipt mechanisms suffice under these tested conditions. Agent ability to choose or interpret those mechanisms was not measured.

A4 Strong-binding overlap and local integration

The two comparisons below preserve the native protections and avoid treating an unavailable abstraction as a failed system. They are local exploratory checks, not a complete external-system benchmark or an efficacy ranking.

Continuity Kernel model probe

The unmodified author model at commit `74841530988f1324bd927da15519486509372917` was called through its native submission interface. The matrix planned 24 records, with context, bound and the CK model for two value relations and four scenarios. Twenty-two records executed; two CK instance-substitution cells were retained as not representable because its exposed abstraction lacks a distinct human-reviewed candidate object. They were not counted as failures, accepted substitutions or zero integrity scores.

All eighteen records in the six jointly expressible cells agreed on coarse admission decisions for normal submission, changed evidence and changed predecessors. CK's Accept/Correction values were adapter encodings, and its evidence validity was a supplied predicate; these are not native demonstrations of the full correction or evidence-byte contract. Fourteen initial rejections had scripted legal recovery witnesses, all twenty-two executable requests replayed without another authoritative change, and eleven controls checked retained protections. The 150 assertions are implementation checks, not independent samples. This probe supports capability mapping, not superiority or absence of instance-binding capability in CK.

Native saved-plan workflow

The application rule was an approval for a named logical saved-plan artifact, with a fresh approval required for another plan. HashiCorp's official automation guidance describes tying approval to the correct plan. OpenTofu treats a supplied saved plan as approved input, so both comparison routes added the same explicit trusted approval record, artifact identifier and digest checks before invoking the unmodified OpenTofu 1.13.1 binary. Those approval checks belong to the workflow wrapper, not a native human-approval API.

One route used saved-plan binding; the other retained the same binding and added explicit correction receipts. Both had the same original proposal, plan and approval information, and the baseline could reconstruct the relation from its logs. Accept and Correction crossed six scenarios and two routes to give 24 main records (twelve route pairs), plus eight controls. The local `terraform_data` fixtures used no cloud resources, external providers, human reviewers or LLM calls.

Both routes satisfied the tested common obligations. Each route's twelve main records used 128 native CLI calls and sixteen approval records, including authorized state advancement in stale-state fixtures. Twelve scripted recoveries succeeded across both routes. All twenty-four main replays were refused as stale without another state change: this is stable non-reexecution, not a returned cached success receipt. The extension generated sixty extra receipt files totalling 40,521 bytes. The 240 checks and new-directory rebuild verify this constructed integration; they do not establish minimal storage cost, production performance or audit time saved.

These comparisons locate the contribution in the explicit four-role correction relation, successor and field-source obligations, and the original online evidence on agent continuation. Identity binding and recovery have strong existing foundations. The OpenTofu mapping did not port the full successor/provenance contract, and neither comparison demonstrated additional execution protection or audit efficiency over a strong binding baseline. A broader system comparison or an audit-task study would require a separately specified observable benefit.

A5 Historical correction-chain audit protocol

The main text reports that an audit of 1,260 historical databases identified 84 retained machine-origin correction chains. This section records the population, discovery rule and per-chain counts so that the claim can be checked from the deposited evidence.

Population and discovery. The audit opened all 1,260 archived databases read-only. Each database belongs to one archived run of the restricted-agent benchmark (three requested configurations, 420 runs each). A correction chain was enumerated from the persisted transition table as a committed transition whose source kind is `AI_SUGGESTION` and whose committed value differs from the proposal value stored on its candidate certificate. No task-success label was used for selection. All such transitions in the population were audited; none was excluded after inspection.

Table 4: Historical correction-chain audit counts. Units are persisted chains and checks, not task successes.

Quantity	Value
Databases scanned	1,260
Correction chains found	84
Chains with an audited violation	0
Chains with no audited violation	84
Behavior-evaluable chains	81
Evidence associations checked	252
Candidate-certificate checks	252
Record versions checked	168
P6 provenance traces checked	168
Chains with unchanged-field checks	0

Configuration counts are $G1 = 29$, $G2 = 26$ and $D1 = 29$; prompt-variant counts are $V1 = 27$, $V2 = 29$ and $V3 = 28$. Terminal classes of the containing runs are 80 `PASS_UTILITY_SAFE`, one `PASS_SAFE_BUT_UTILITY_FAILED`, two `MODEL_API_FAILURE` and one `TIMEOUT`. All 84 chains are single-field, so the audit supplies zero nonvacuous unchanged-field checks; copy-forward is supported by the controlled multi-field evidence, not by this audit.

What the audit does and does not establish. It verifies that the four roles can be reconstructed from persisted relations and archived evidence, and that the recovered relations satisfy the audited binding. It does not show that the historical system enforced the relation at run time, because the audit reconstructs from persisted state rather than from an enforcement log; it does not establish natural mismatch prevalence or adoption need. Chain identifiers and input hashes are retained in `research/authorization-granularity-phase1-2026-10-06/` (`correction-chains.jsonl`, `correction-summary.json`, `audit_corrections.py`); the receipt association pairs tool calls and results by tool name in archived event order because the archive has no uniform call identifier.

A6 Controlled-case summaries and artifact mapping

The main text reports outcomes of the controlled comparisons and the bounded formal checks. Those counts are summarized here with their denominators and with the artifact paths that retain the per-case records.

The E1 and E2 records are retained in the pinned DKE-supplement archive cited in the main text; this addendum does not re-analyze them. The labels “legal”, “rejected” and “display-only” follow the submitted protocol. All cases are constructed or scripted, not production interactions.

Table 5: Controlled and formal evidence: counts, denominators and units.

Evidence	Counts	Unit and artifact
E1, exact journal and relational reference	165 cases each: 45 legal accepted, 120 rejected (the 15 equal-value substitutions are rejected)	Two exact realizations; 495 executions in total across three arms. `results/e1/inputs.json`, `results/e1/receipts.jsonl`
E1, context journal	165 cases: 60 accepted (45 legal plus 15 equal-value substitutions), 105 rejected	Same constructed cases under the retained-context policy.
E2, per path	30 cases: 9 legitimate, 9 request substitutions, 9 stale/replay/interrupt, 3 display-only	Scripted browser interactions plus 6 replay setup confirmations. `results/e2/browser-observations.jsonl`
Rejection audit	372 rejected attempts across E1 and E2	Attempts, with expected cause and unchanged fact-database digest. `rejection-audit.json`, `verification.json`
Bounded checks	36 commands in each of two scopes (72 outcomes); 9 projections SAT; 20 selected mutants UNSAT	Command outcomes, not independent proofs. `research/authorization-granularity-phase1-2026-10-06/formal/`

A7 Refreshed-preview exposure in the N family

The main text reports that no N transition substituted an instance without authorization. This section records how often an equivalent but distinct candidate was exposed and which legal continuation each agent chose.

Population and decision rule. The analysis unit is the N arm that reached review and has a retained state record: 127 arms in total (original A six, original B 63, supplementary A 58). Exposure is judged relative to the review object that existed before the refreshed preview, not relative to every grant present in the final state. For each arm we take the earliest authorization by issue time, require both candidate contexts to be present dictionaries, and compare their canonical JSON serialization; a preview candidate counts as an equivalent alternative when its identifier differs from that review object and its canonical context is equal. A candidate that is previewed and later separately authorized therefore still counts as an exposed alternative; treating it as part of the review set would understate the denominator. Presentation is verified from the saved model-input files, which contain the exposed candidate identifier, together with a later model-response event without an error. Commit paths are read from persisted transitions and checked against the reviewed candidate of their authorizing grant. The recount reads archived states, events and saved model inputs and makes no new model calls.

Table 6: Refreshed-preview exposure in the N family. "Reviewed" counts arms that reached review and retain a state record. "Exposure" requires a preview candidate with a different identifier and a canonically equal retained context, judged against the pre-preview review object. "Presented" requires the candidate in the saved model inputs and a later non-error model response. "Reused" committed the reviewed instance; "Reauthorized" requested a fresh authorization for the previewed instance and committed it. No commit mismatched its authorizing grant.

Cohort	Policy	Reviewed	Preview	Exposure	Presented	Reused	Reauthorized
Original A	Context	3	3	3	3	3	0
Original A	Bound	3	3	3	3	3	0
Original B	Context	32	30	30	30	30	0
Original B	Bound	31	30	30	30	30	0
Supplementary A	Context	29	29	29	29	29	0
Supplementary A	Bound	29	29	29	29	28	1
Total		127	124	124	124	123	1

Across the cohorts, 124 of the 127 analyzable arms received a preview of an equivalent but distinct candidate, and in all 124 the candidate and a later non-error model response are present in the saved inputs.

Of these, 123 committed the reviewed instance and one supplementary-A bound arm (`FORMAL-N-016-A`) requested a fresh authorization for the previewed instance and committed it. No commit was observed whose candidate did not match its authorizing grant. The observation therefore shows two legal continuity-preserving behaviours under an available alternative, not a lack of opportunity; it does not estimate how often such an alternative arises in production. The three reviewed arms without a preview are retained in the counts rather than dropped.

One original-B context arm (`FORMAL-N-020-B`) has no final state; the last numbered after-state is used. That record supports the observed events up to its save point (preview, model response, commit and persisted transition are present) but is not treated as a complete final state, and the original frozen integrity and termination classification are unchanged. Excluding this arm as a sensitivity check leaves 126 analyzable arms, 123 equivalent exposures, 122 reviewed-instance commits and one reauthorization.

The 124 arm exposures cover 64 actual cohort/configuration pairs and 32 unique task definitions: 60 pairs expose both arms and four expose one. The four one-arm cases are original-B N-005, N-020, N-023 and N-026. The 186 planned arms correspond to 93 pairs (32 original A, 32 original B and 29 supplementary A). Repeated configurations and shared prefixes do not create additional independent tasks. The identifier-based recount is retained in `tables/integrated-n-exposed-pairs.csv`.

Source records. Counts were read from the archived states, events and saved model inputs in `revisions/2026-10-07-kais-continuity/replication-package.zip` (original collection; SHA-256 `1bc17338d18f198811e0e58eca9b64bded29f9b01cd61558862455e7e0d469d8`) and `research/phase2-results-snapshots/2026-10-07-complete/quota-recovery-complete.zip` (supplementary A; SHA-256 `536f8b7d784d1af118145affd90c3d613b67ad44d9c5b3607b463bcebeacbec1`). State paths follow `formal-results/FORMAL-N-*/<policy>/state/final.json` and `results/FORMAL-N-*/attempt-001/<policy>/state/final.json`; presentation uses the adjacent `model-inputs/` and `events.jsonl` records. The recount script and per-arm output are retained in `revisions/2026-10-18-kais-recalibrated/qa/n_exposure.py` and `revisions/2026-10-18-kais-recalibrated/qa/n-exposure.csv`; the recorded per-arm flags include whether an after-state was used and whether any unauthorised substitution was observed. The analysis exports in Online Resource 1 report 30 N transitions per policy for original B, which matches the 30 observed commits per policy recounted here. An earlier draft of this section judged exposure against the final grant set; that rule is superseded because it excluded a previewed candidate that was subsequently authorised.

A8 Configured ToolGate and public-tool reference workflows

Comparison target and source versions. The question is whether a separately developed contract-execution framework can implement the shared correction obligations and pass its committed state to a subsequent workflow. ToolGate is the method of Liu et al., *ToolGate: Contract-Grounded and Verified Tool Execution for LLMs*, Findings of ACL 2026, pp. 9653–9684 (`https://doi.org/10.18653/v1/2026.findings-acl.470`). The paper links to `https://github.com/OceannTwT/ToolGate`, which redirects to the recorded source `https://github.com/Liam-magicbook/ToolGate`. We use commit `976ad3f5528f89e248cbdf55840347b445934155`, a snapshot preceding final publication, and retain its source files unchanged. This identifies the implementation tested; it does not imply reproduction of the final paper's model experiments. The public Retail tools and tasks come from `https://github.com/sierra-research/tau2-bench` at commit `dd8757d4c8791dad7b3cdaf02a105a3d5f9d86bd`.

Table 7: Implementation ownership in the comparison. Both routes receive the full shared business requirements; T does not call the C verifier.

Layer	Configured ToolGate (T)	Contract backend (C)
Existing implementation	Original pipeline: precondition, tool, post-condition, symbolic-state update	Frozen `ReviewForms` and `SqlAlchemyFormRepository` admission/persistence backend
Business adaptation	Author-written callbacks for candidate/authorization binding, value, predecessor and successor/source relations	Input construction and mapping into the unchanged contract backend
Persistent boundary	Author-written candidate, grant, version, source and transition records; SQLite commit bridge	Existing persistent backend through `stores.Store("full", ...)`
Shared execution harness	Native Retail tool on a cloned complete world; deterministic review and recovery; independent read-only scoring of actual SQLite and complete reference-world state	

ToolGate's paper describes contracts extracted from tool specifications and documentation. Here the application predicates are supplied manually through its normal Python callback interfaces. The original pipeline runs with `llm_client=None`, a scripted reasoner and fixed single-tool retrieval; no model-generated decisions or LLM verifier calls are measured. All shared relations are available to the baseline, so the initially proposed labels `T-configured` and `T+relation` denote one condition. The additional tables and commit bridge are author contributions to the adapter, not native ToolGate external-transaction guarantees.

For T, the bridge begins a SQLite transaction, reads the predecessor, runs the original pipeline and writes only after successful postcondition checks. Tools produce a candidate in an isolated clone rather than committing an external effect before verification. For C, the original backend makes the admission decision. The shared projection covers candidate/authorization association, proposed and authorized values, predecessor, complete successor and unchanged-field sources; C's additional producer/evidence metadata are not claimed to have been equivalently tested. The scenarios are sequential: arbitrary external-side-effect rollback and concurrent freshness are outside this execution comparison.

Development controls. Eight constructed controls are kept separate from the public-workflow denominator. D01–D04 cover unchanged approval, corrected-value execution, reuse after an alternative preview exists, and fresh authorization of the new instance. D05–D08 cover a new instance under the old grant, a predecessor advanced by another transaction, execution of the uncorrected proposal, and an unauthorized change to an unchanged field. Both routes satisfy 8/8 expectations: four authorized commits and four attempts without a new business commit. The independently advanced predecessor in D06 is retained. These are development controls, not held-out benchmark cases.

Public workflow selection and intervention. Of fourteen eligible tasks unused in the earlier three-task comparison, twelve groups remain after deduplicating the common initial world and full reference action sequence with action identifiers removed. The six smallest representative IDs—59, 71, 78, 96, 101 and 102—were selected before this run. Some still share orders or actions and are not independent natural samples. Each is crossed with four fixed scenarios and two implementations: $6 \times 4 \times 2 = 48$ routes, or 24 per implementation.

Table 8: The four controlled scenarios applied to each public reference workflow. Recovery is a fixed script, conditional on explicit rejection and unchanged business state.

Scenario	Initial request	Predefined continuation
Corrected	Execute the original candidate at the corrected, authorized address	Run the reference suffix after commit
Reauthorized	Execute a new equivalent candidate under its fresh grant	Run the reference suffix after commit
Instance mismatch	Execute the new candidate under the original grant	Reuse the original reviewed candidate, then run the suffix
Wrong value	Execute the proposal address rather than the authorized correction	Retry at the authorized value, then run the suffix

The proposal appends the fixed marker `[unreviewed-draft]` to the reference address's second line; the authorized value is the original reference action's address. Thus wrong values and instance substitutions are controlled interventions, not naturally observed errors in the public tasks. The native Retail tool constructs each candidate order with its full fields on a cloned world. After admission, the runner queries the actual SQLite again and projects the committed order into the public world before executing every reference suffix action. Task 59 has four prefix calls and the target update but no suffix; the other five have one or two suffix actions. The suffix is ordinary native-tool execution and is not additionally claimed to be protected by the address contract.

Scoring and complete results. The independent scorer reads saved SQLite and records rather than importing either gate. It checks the first attempt and recovery separately, candidate/grant/transition links, unchanged fields and sources, the persisted state passed to the suffix, and equality of the complete `products/users/orders` world to native reference replay. Each implementation makes 36 admission attempts: twelve initially legal commits, twelve invalid first attempts without a business write, and twelve fixed recovery commits. Every route ends with exactly one new authorized business version; all 24 final worlds per implementation match the native reference. There are no engineering errors. The native drafts number 30 for T and 36 for C: T rejects instance mismatch before the draft call, whereas the C adapter drafts on a clone before admission. Each implementation makes 32 suffix calls. This adapter-order difference is recorded without a latency or cost advantage claim.

The frozen strict scorer reports T = 23 PASS/1 FAIL and C = 24 PASS/0 FAIL. Its only failure is `P59-corrected/T-configured`: two first-import initialization messages violate the requirement that

captured output be empty. Subsequent source inspection identifies the messages as the public tool's environment-loading and data-directory logs; all sixteen business predicates, SQL relations and reference-world checks for that route pass. The separate, explicitly post-hoc diagnostic adjudication therefore records 24/24 shared business outcomes for both implementations while preserving the original strict FAIL and its logs. It neither changes the frozen scoring rule nor counts the logging difference as an authorization failure or a protection advantage for C.

These results establish implementation agreement on the tested shared obligations and continuity of scripted reference workflows. They do not establish statistical equivalence. The public task dialogue, `NL_ASSERTION` checks and official evaluator were not run; reference-world agreement is accordingly distinct from official benchmark success. Likewise, fixed recovery is not an observation of autonomous agent recovery or human review burden. The separate online study in the main text remains the evidence for agent continuation.

Artifact locations. The development protocol, `UPSTREAM-SOURCE.json`, `FREEZE.json`, adapters and independent scorer are in `research/kais-toolgate-feasibility-2026-10-20/`; the saved run is `runs/20261020T121403449662Z/`. The public protocol, selected-input manifest, source freeze and independent scorer are in `research/kais-toolgate-public-reference-2026-10-20/`; its run is `runs/20261020T123319655459Z/`. Each run retains `RUN.json`, `SCORE.json`, per-route CSV, original records and SQLite databases. The public directory also retains `DIAGNOSTIC-ADJUDICATION.md`; the frozen strict score has SHA-256 `39a1d65be1b47abfdb189c57a7287aba8ac66e256688c697ee7ee705293b60bc`. The unchanged public source bytes and task provenance are pinned by `research/kais-external-baseline-sprint-2026-10-15/vendor/pilot/source/manifest.json`. Both experiment directories provide read-only rescoring instructions in their `README.md`; no new model calls are needed.

A9 Extended specification and characterization

This section retains the conditions and derivations accompanying the specification in the main manuscript. The statements concern the declared observation model and execution contract. They introduce no additional experiment or change to the original or supplementary collection.

A9.1 Observation domain and local outcome

The observation model in Online Resource 1, Section S2, concerns one declared admission obligation and the finite history produced for it. It permits malformed outcomes, including a declared batch fragmented across connected commits. Candidate and authorization handles are opaque and equality-preserving: a projection cannot recover omitted information by decoding a handle.

Let

$$D = \{D_C, D_V, D_F, D_B, D_S\}$$

denote the five logical information classes. For a history h, $\pi_I(h)$ is its projection onto the classes in $I \subseteq D$. The observations expose:

D_C**:** Candidate and authorization-target handles and non-temporal context.

D_V**:** Proposed, authorized, and committed values with their role labels.

D_F**:** Expected and observed predecessor discriminators.

D_B**:** Declared and actual effect domains, commit grouping, and successor count.

D_S**:** Field-source anchors and local resolution, consistency, and copy-forward checks.

These are logical projections, not five required physical storage objects.

Write C, V, F, B, and S_{local} for the checks computed from these observations. C checks the target record and equality of the submitted candidate and authorization target. V checks explicit human-authorized attribution and equality of authorized and committed values. F compares the predecessor discriminators. B requires equality of the declared and actual effect domains and exactly one successor. S_{local} checks unique source resolution, field/value consistency, and exact source retention for unchanged fields.

The modeled local outcome O_{obs} is a triple comprising status, successor category, and trace category. Its status is admissible exactly when

$$C \wedge V \wedge F \wedge B \wedge S_{\text{local}}$$

holds. Its successor category is fragmented when the successor count differs from one, partial when there is one successor but the actual and declared effect domains differ, and complete otherwise. Its trace category is complete exactly when S_{local} holds. These are the categories computed by the executable observation model. They do not encode runtime interruption, reconstruct the full successor value map, or certify the deep authorization–candidate–evidence chain.

The operational gate has separate inherited authorization inputs: principal and policy epoch, validity window together with admission time, grant and idempotency state, and the transaction envelope. The characterization below does not assert that the five observations alone determine every operational outcome.

A9.2 T1: an irredundant basis for the modeled local outcome

T1a: necessity in the declared observation model. For each $d \in D$, the witness construction, retained in the v15 equality-corrected derivative (Online Resource 1, S2.6–S2.8), gives two well-formed histories h_s, h_u such that

$$\pi_{D\setminus\{d\}}(h_s) = \pi_{D\setminus\{d\}}(h_u), \qquad O_{\text{obs}}(h_s) \neq O_{\text{obs}}(h_u).$$

The checker verifies these properties for the constructed pairs. Every proper subset of D is therefore insufficient to determine O_{obs}: choose an omitted class and its witness pair, then take the corresponding sub-projection. Equality on the larger retained projection entails equality on that sub-projection, while the outcome labels remain different.

T1b: sufficiency for the modeled local outcome. For any two well-formed histories in this observation model,

$$\pi_D(h_1) = \pi_D(h_2) \implies O_{\text{obs}}(h_1) = O_{\text{obs}}(h_2).$$

Observation and history-domain comparisons use sorted-key, whitespace-minimal canonical JSON serialization, preserving JSON value types. JSON predecessor discriminators use this convention in both D_F and F; opaque handles retain identity equality. Each of the five checks is a function of its corresponding observation. The successor category is a function of D_B, and the trace category is a function of D_S. Substituting these functions into the definition of O_{obs} establishes the implication. This is a result about the declared local classification, rather than a proof of arbitrary runtime behavior. Online Resource 1, S2.8 documents the historical checker's departures from this equality convention, the v15 repair and 38 separate offline regressions. These finite checks support the constructions; the general implication follows from the factorization above.

T1c: irredundancy. Together, T1a and T1b establish that D is sufficient and that no proper subset is sufficient for O_{obs} in the declared observation model. This is an irredundant observation basis, not a minimum-cardinality schema, a unique representation, or completeness over all admission failures. The inherited authorization parameters are separate inputs to the operational gate; the witness checker does not validate their runtime behavior.

A9.3 T2: equal endpoint values with different continuity

For an issued authorization, let c_r be its reviewed candidate and c_s the submitted candidate. Retained context $K(c)$ is the declared non-identity review information. Define

$$K_{\text{eq}} \iff K(c_s) = K(c_r), \qquad I_{\text{eq}} \iff \text{id}(c_s) = \text{id}(c_r).$$

G collects the shared disclosed checks and excludes an implicit instance comparison. The policies are

$$\begin{aligned} \text{Admit}_{\text{context}} &= G \wedge K_{\text{eq}}, \\ \text{Admit}_{\text{bound}} &= G \wedge K_{\text{eq}} \wedge I_{\text{eq}}. \end{aligned}$$

T2a: refinement. $\text{Admit}_{\text{bound}} \Rightarrow \text{Admit}_{\text{context}}$ by conjunction elimination. Its interpretation is narrower admission under the specified instance requirement.

T2b: difference set and paired executions. On a common request-state domain, define

$$S = \{r : G(r) \wedge K_{\text{eq}}(r) \wedge \neg I_{\text{eq}}(r)\}.$$

Then the difference between the context-admitted and bound-admitted sets is exactly S. Strictness on a given domain requires that this difference contain a reachable request.

To compare committed results, take two independent copies of the same pre-admission state, including an unconsumed grant and the candidate registry. In the first copy, context admits (c_s, x_a) under that grant. In the second, bound admits (c_r, x_a) under the corresponding copy of the grant. These are alternative executions, not sequential consumption of one grant. Both install x_a at the same field and copy all other values and exact prior sources forward, so their field-wise value maps agree.

For the fixed-registry identity-isolation construction, all non-identity candidate components are equal. After consistently renaming the newly allocated transition atom, the local projections on $D \setminus \{D_C\}$ also agree. The candidate-binding observation differs: the context execution violates the declared instance requirement while the bound execution satisfies it. Holding predecessor state and final value fixed therefore does not determine review continuity. An execution rule can distinguish the two cases; recovery, cost, and completion remain empirical questions.

T2c: link to T1. The archived fixed-registry identity-isolation witness instantiates this paired comparison. It holds the candidate pool, authorization, non-identity content, and the other four observations fixed while changing the submitted candidate. D_C exposes a distinction that endpoint values and the other local observations do not.

T2d: request-domain and execution-domain coincidence. For a declared request-state domain Q, the predicates coincide on Q exactly when $S \cap Q = \varnothing$. This holds, for example, if each reachable context class contains at most one candidate, or if the submission interface admits only the reviewed candidate under its grant.

A different condition concerns the actual submitted requests $T \subseteq Q$. If every submission uses an authorization whose reviewed target is the submitted candidate, then $S \cap T = \varnothing$ even when $S \cap Q$ is non-empty. Merely having another authorization available is insufficient: the submission must use the matching authorization. A trace in which all actual submissions follow that discipline does not establish equality of the predicates over all possible requests.

A9.4 Typed transitions and policy-relative satisfaction

The paper uses P2 for context/evidence integrity and P3 for principal authority and the authorized value throughout. Cross-candidate context agreement is a separate binding condition; it is not the internal integrity condition P2.

Let Σ contain the current version, complete field-value and field-source maps, and the candidate, authorization and transition registries. A submitted item is $u = (c_s, a, x)$, where $a = (c_r, x_a, q)$. For the singleton presentation, a persisted transition has type

$$\begin{aligned} \tau &= \langle j, r, f, e, v^-, v^+, x, p, \mathrm{id}(c_s), \mathrm{id}(a) \rangle, \\ \tau &\in \mathsf{Tid} \times \mathsf{Record} \times \mathsf{Field} \times \mathsf{Evidence} \times \mathsf{Version}^2 \\ &\qquad \times \mathsf{Value} \times \mathsf{Producer} \times \mathsf{Cid} \times \mathsf{Aid}. \end{aligned}$$

The transition identifies the executed candidate; the authorization separately identifies the reviewed candidate; Cid and Aid are registry keys. Certificate rows in the realization resolve these references to immutable candidate records. The fields of this tuple are observable relations, not an assumption that an arbitrary stored tuple is valid.

For $b \in \{\mathrm{context}, \mathrm{bound}\}$, define the binding condition

$$\begin{aligned} \beta_{\mathrm{context}}(u) &\iff K(c_s) = K(c_r), \\ \beta_{\mathrm{bound}}(u) &\iff \beta_{\mathrm{context}}(u) \wedge \mathrm{id}(c_s) = \mathrm{id}(c_r). \end{aligned}$$

Thus P1 denotes β_b under the selected policy. Under bound admission it includes instance equality; under context admission it requires only the declared context equality. In particular, context admission does not borrow instance equality through another shared check.

Write $z = (\Sigma^-, u, \tau, \Sigma^+)$ for a committed outcome. The satisfaction relation is defined by the following checks on the outcome and the validated admission inputs:

P1$_b$: $\beta_b(u)$ holds for the submitted and reviewed records.

P2: The declared record, field, evidence identity/locator and expected predecessor agree with the submitted candidate. The evidence belongs to the target record; the transition retains the candidate's record, field, evidence and producer. This concerns internal consistency, independently of P1$_b$.

P3: The authorization comes from a permitted principal at commit entry, is usable for this admission, and explicitly records x_a. The submitted, transition and newly installed field values equal x_a under the declared comparator; no equality between x_a and the retained proposal x_c is required.

P4: The candidate's expected predecessor equals the authoritative version observed in Σ^- within the admission transaction.

P5: Exactly one successor of that predecessor is installed. Its changed fields and new transitions correspond exactly to the admitted items, each transition records the actual predecessor and successor, and all other values are retained. The successor, new transitions, source updates and grant consumption belong to the same commit unit.

P6$_{\mathrm{local}}$: Each changed field points to its unique new transition, with the same field and installed value. Each unchanged field retains its exact prior source. Source references resolve uniquely in the retained transition registry.

For a batch, u and τ are replaced by the admitted item set and its transition set; P1–P4 apply itemwise and P5 requires one shared record/version envelope and an item–transition bijection. After initial coverage, the concrete admitted set contains exactly the submitted fields whose canonical values change and must be nonempty, as specified in Online Resource 1, S2.

Formally, with $L(z)$ denoting P2–P5 and $\mathrm{P6}_{\mathrm{local}}$ together,

$$z \models \mathcal{R}_b^{\mathrm{local}} \quad \Longleftrightarrow \quad \beta_b(u) \wedge L(z).$$

P0 restricts the execution model: candidate-producing operations cannot write authoritative values, versions, sources, transitions or authorizations. It is not inferred from $z \models \mathcal{R}_b^{\mathrm{local}}$. The pre/post relation specifies one complete step; crash atomicity cannot be inferred from endpoint snapshots. Trusted immutable lookup, canonicalization and atomic installation are explicit execution premises. The shared gate G implements the common checks and lifecycle constraints; its definition must not hide the bound-policy conjunct in β_b.

Full P6 also checks the source–transition–authorization–candidate–evidence chain, including retained references, field/value/producer/version consistency and source-version uniqueness. Its binding link is interpreted under the declared policy: a context-authorized transition retains both candidate roles and requires β_{context}, whereas an exact-instance trace also requires β_{bound}. A context-legal substitution can therefore satisfy its local contract while failing the separate exact-instance trace criterion. The positive formal preservation result below is for the bound model only; this policy-relative definition adds no new model-checking result.

A9.5 Paper obligations and archived Alloy assertions

Table 9 is a semantic crosswalk, not a renaming of the frozen source or a claim that every paper-level premise was model checked. H denotes the historical singleton model `formal/alloy/auto_decte.als`; B denotes the batch model `latest/formal/alloy/batch/auto_decte_batch.als` in the fixed reference artifact cited in Online Resource 1, S1. The B command manifest uses freeze `2026-09-11-trace-witness`. Assertion names retain their historical spelling and numbering: neither source has an assertion named P2.

Table 9: Paper obligations mapped to exact archived assertion names. H and B are source namespaces, not additional evidence counts.

Paper relation	Archived assertions	Encoded part and limit
P0	H: `P0_no_machine_write`; B: `P0NoMachineAuthoritativeWrite`	Frame modeled machine operations. They do not prove that a deployed caller cannot bypass the gate.
$\mathrm{P1}_{\mathrm{bound}}$	H: `P1_auth_nontransfer`; B: `P3AuthorizationIntegrity`	Canonical certificate equality binds authorization to the submitted certificate. The candidate abstraction must preserve this relation.
$\mathrm{P1}_{\mathrm{context}}$	No separate context-policy assertion	Defined here through K_{eq}. Neither Full model verifies the relaxed policy or the three-level binding comparison.
P2	H: `P1_no_cross_context_admission`; B: `P1ExactContext`	Item/certificate record, field, evidence and expected-version consistency. These historical "P1" names correspond to paper P2, not reviewed-instance equality.
P3	H: `P3_authorization_reference_integrity`; B: `P3AuthorizationIntegrity`	Principal membership, explicit authorized value and reference checks. Dynamic revocation, expiry and grant consumption are outside these models.
P4	H: `P4_stale_reject`; B: `P4Freshness`	Expected/current predecessor equality. Database concurrency semantics are an implementation obligation.
P5	H: `P5_transition_integrity`; B: `P5TransitionBijection`, `FullNoPartialItemEffects`, `RejectionHasNoEffect`	Transition relations, complete batch effects and rejection framing in atomic pre/post steps; no crash or database atomicity proof.
P6	H: `P6_trace_soundness`, `P6_missing_source_incomplete`, `P6_duplicate_incomplete`; B: `P6MissingAnchorIncomplete`, `P6DuplicateSourceVersionIncomplete`, and positive preservation (below)	Missing/duplicate-source diagnostics and conditional positive trace checks; scope and pre-state restrictions remain essential.

The B authorization assertion calls `itemAuthorizationOk`, which includes both `authorizationCertificateBinding` (paper P1) and `principalBinding/authorizationValueBinding` (paper P3). Its context assertion calls `itemContextOk` (paper P2). This explains why the mapping is many-to-many. The

Full assertions often check consequences of explicit contract conjuncts; effective ablation witnesses and independent persisted-state projections provide different evidence and are not subsumed by these names.

The positive B P6 assertion, `LegalAdmissionTraceCompleteUnderWellFormedPre`, assumes every anchored predecessor field has a complete trace and no prior transition targets an admitted field's next version. It concludes trace completeness for admitted fields after one legal Full step. The same archive includes reachable initial and carry-forward precondition witnesses. These checks do not establish arbitrary-history induction, every unchanged field's full chain, external-document immutability, or the correctness of an unmodeled review channel.

For source identification, SHA-256 of H is `3e42226a642e458170b85671a56ff8438fdc7c8dba72b970c53822867e7f9d19`. The B S1 and S2 source hashes are, respectively, `e5275f855eff19469ecb73da26d1f3dab383c2578e72f4b526a9694230f8e70b` and `3bf12ffb44259fff5022e749fe13d2573ae1539e7f8306a3402da1db0007b947`. The matching manifest records 72 outcomes (22 SAT witnesses and 14 UNSAT checks per profile). This crosswalk identifies the archived results; it adds neither a new Analyzer run nor an unbounded proof.

A9.6 T3: conditional gate correctness

Let $\mathcal{R}^{\text{local}}_{\text{bound}}$ be the transition contract in Section A9.4. It combines P1 (instance binding), P2 (context and evidence integrity), P3 (principal and authorized value), P4 (freshness), P5 (complete atomic successor), and the local part of P6 (source relations). P0 is an execution-boundary assumption, not a predicate on one successor. Full trace soundness additionally concerns the deeper authorization–candidate–evidence chain. Section A9.5 distinguishes these paper obligations from the historical Alloy assertion names.

Conditional gate soundness. Assume direct-write exclusion, a trusted gate and persistence adapter, trusted lookup of content-immutable candidate records preserved to consumption, authority checked at commit entry, consistent canonical equality, transaction-scoped reload and predecessor validation, and atomic installation of the transition, authorization consumption, complete successor, and source records. Candidate creation must not permit overwriting a retained candidate or its review linkage; this storage prerequisite is separate from P0's exclusion of bypass writes to authoritative state. If the bound gate commits an outcome z, then $z \models \mathcal{R}^{\text{local}}_{\text{bound}}$: each listed relation is validated and the same validated objects are installed atomically. This conditional argument uses the explicit relations above; it does not infer soundness from the gate's returned status alone.

Extending this conclusion to full bound-policy P6 requires a trace-complete predecessor, valid and immutable referenced records, validated new chain links, and uniqueness of the successor's transitions. An independent audit checks those relations in the realized state. Audit alone is not a premise that makes an unchecked commit sound. The archived positive preservation assertion checks admitted fields in one step under its explicit pre-state conditions; it is not an induction over arbitrary histories or a check of the context policy.

Relative completeness. If r is contract-legal with a non-empty change, the gate's observed state equals the authoritative state at validation, the canonical forms match, and the key is fresh and the grant unconsumed, then

$$\text{Admit}_{\text{bound}}(r, \Sigma_v) = \text{yes}.$$

This is a conditional acceptance result; it does not classify every rejection. Legal no-ops are excluded by the non-empty-change requirement. Formal and implementation models can differ on same-value admission. Soundness and completeness are relative to the declared equality relation, and disagreement between review semantics and implementation equality can cause false acceptance or rejection. The controlled implementation and bounded mapping evidence supports the realization, rather than replacing the argument's assumptions.

A9.7 T4: continuation obligations and available capabilities

Let Cap describe the available continuation capabilities. $\text{Recover}(\Sigma, \kappa, \text{Cap})$ means that a finite continuation generated using those capabilities can reach a successful bound admission whose candidate matches the authorization used for that admission. This authorization may be the original grant or a matching grant obtained during the continuation.

Table 10 states continuation obligations, subject to all remaining admission checks. The rows are not unconditional sufficient conditions.

Under instance binding, the existing authorization covers its reviewed candidate. Executing any different candidate requires an authorization matching that candidate. This distinguishes reuse from matching

Table 10: Continuation obligations following rejection or uncertainty.

Failure or uncertainty	Continuation obligation	Decision burden
Instance mismatch; otherwise unchanged valid context	Reuse the original reviewed candidate under its still-valid grant, or obtain/use an authorization matching the submitted candidate.	Re-binding for the fixed-value identity-only case
Stale predecessor	Construct a candidate against current state and obtain matching authorization.	Re-decision
Changed evidence	Construct a candidate using current evidence and obtain matching authorization.	Re-decision
Consumed grant with a prior commit	Retrieve or confirm the prior receipt; this confirms an effect rather than creating a new one.	Confirmation
Unconsumed but no-longer-valid authorization	Obtain a currently valid matching authorization, if permitted.	Authorization renewal
Principal or policy denial	Restore applicable authority or obtain authorization from a permitted principal.	Escalation
Unknown commit outcome	Obtain evidence distinguishing committed from uncommitted before choosing retry or renewal.	Diagnosis

authorization; it does not restrict legal sequences to two candidates, since a third candidate can be proposed and authorized. Recover quantifies over state and capabilities, not over the agent's knowledge. Existence of a continuation is distinct from the agent attaining it. Receipt confirmation, in particular, concerns an already committed effect rather than a new successful admission.

A9.8 Schema connection

The specification is packaged as four role objects, five observation classes, obligations P1–P6, and the policy parameter. The candidate tuple excludes implementation-level certificates; certificates are derived artifacts of the realization.

In the running c_{17}/c_{18} example, bound rejects c_{18} under c_{17}'s grant. Subject to the remaining checks, it permits the reviewed candidate under the existing grant or another candidate under matching fresh authorization. Context permits the equivalent c_{18} request under the original grant, while the independent instance-continuity oracle records a violation. Admissibility under a policy and conformity to that independent oracle are different judgments.

A10 Expanded design-selection guidance

This section preserves the design axes, selection questions, matrix, and principles underlying the compact guide in the main manuscript. The workflow owner supplies the responsibility rule. The guide derives implementation and recovery choices from that rule and the evaluated mechanisms; it does not establish which rule an organization should adopt or add a developer-study result.

A10.1 Binding, mismatch handling, and the enforcement boundary

The first axis is the bound object: no gate, value-bound, context-bound, content-bound, or instance-bound. The gated policies share G. When the content field set R extends the retained context field set K, equality on R implies K_{eq}. Under trusted lookup of content-immutable candidate records, I_{eq} implies equality on R. Writing $P_{\text{value}} = G$ for the value-only policy gives

$$P_{\text{bound}} \subseteq P_{\text{content}} \subseteq P_{\text{context}} \subseteq P_{\text{value}}.$$

In the declared contract, K spans every non-identifier candidate field, so context and content binding coincide; they separate when retained context omits a decision-relevant field. External document contents require a separate check at consumption. This is a chain of admissible request sets under those choices, not a universal ordering of unrelated authorization systems. Reauthorization-required is not the next point on the chain. At the level of one admission predicate it shares the instance-bound decision and differs in the failure branch. It changes the continuation and reachable next states, rather than the admitted set of that single admission.

The second axis is mismatch handling: automatic adoption with a substitution record, unexplained denial, denial with actionable feedback, or routing to reauthorization. The context policy tested here uses automatic adoption with a substitution record: it admits the substitute without a rejection and records

the executed candidate in the transition. Recording a substitution does not itself make that substitution conform to an instance-bound responsibility rule.

The third axis is the anchor and timing: a request, server-held review record, or persisted constraint may be compared at review, handoff, commit, or grant consumption. An engineer must identify which authoritative object the check reads and whether that check governs the eventual persistent effect.

A10.2 Interpreting existing mechanisms through the axes

The mechanisms compared in the main manuscript occupy different locations on these axes. Gerrit and GitLab make review applicability explicit when a patch set changes; OpenTofu's saved-plan mode retains the decisions supplied to apply. Continuity Kernel binds a proposal and a sealed candidate, and ToolGate permits application relations to be expressed as pre/postconditions. Their cited descriptions and the obligation-level comparison remain in the main manuscript.

These examples motivate making the binding object a declared application choice. The correction relation specifies which candidate a corrected authorization covers and joins that choice to the installed successor and its sources. In the configured ToolGate adaptation, application callbacks express the relations and the commit bridge supplies the external persistence boundary. This is the implementation route to inspect when deciding whether a symbolic postcondition also protects durable effects.

A10.3 Selection questions and matrix

Short selection sequence.

1. What responsibility rule has the workflow owner declared: value/content approval or approval of the reviewed artifact, and which carryovers are permitted?
2. Can an equivalent alternative be re-materialized by preview, handoff, or replay?
3. If binding fails, does the agent have a usable object or channel: reuse, fresh authorization, state read, or confirmation?

If there is no legal continuation channel for the required responsibility rule, stop that submission path and build the channel first. Do not silently downgrade to context. Choosing context instead is a change in the responsibility rule, not a fallback that preserves the same instance-bound rule.

The selection also depends on identifier stability, the location of the authoritative review record, which proposed and authorized values and grants are retained, and who may authorize a replacement candidate.

Reversibility affects the cost of a mistake; it does not by itself determine what a grant authorizes. The no-reachable-alternative condition in Table 11 requires an interface guarantee rather than an observed zero substitution count.

The main manuscript's release-configuration example illustrates two declared rules, not two tested deployments. Under an artifact rule, installing a new record with identical content can violate the required link between review and installation; a substitution log alone does not authorize approval transfer. Under an explicit content-carryover rule, matching the approved content, evidence and predecessor may permit that transfer. These examples explain the consequences of a chosen rule, not the frequency or severity of such cases in practice.

When both reuse and review channels exist, the binding rule does not determine which legal route the agent will choose. In the online study, review immediately grants any valid candidate. In deployment, a review request may wait, be refused or require transfer to another reviewer. Queue waiting and human re-inspection are unmeasured costs, separate from the reported agent-side calls and suffix times. The fixed observed count of eight fresh-authorization routes does not predict how often a human-reviewed deployment would choose that route.

A10.4 Principles and implementation patterns

1. Retain the machine proposal and the authorized value as distinct persisted roles.
2. Retain the reviewed candidate and its grant, not merely a record that review happened.
3. Bind execution to the reviewed candidate where responsibility attaches to the artifact, and state the cost.
4. Make rejection actionable by reporting the failed relation and the legal next step.

Table 11: Implementation and recovery choices given the declared responsibility rule. Every new admission must pass the remaining checks; reversibility is not an authorization condition.

Responsibility and current validity	Available channel	Starting choice
Content or value; equivalent alternatives allowed	Any	Value-, context-, or content-bound; declare compared fields
Reviewed artifact; no reachable distinct-instance alternative	Any	Predicates coincide on that reachable domain
Reviewed artifact; original candidate and grant remain valid	Reuse	Instance-bound; submit the reviewed candidate, even if the operation is irreversible
Reviewed artifact; original state, evidence or grant is no longer valid	Review	Prepare a currently valid candidate and obtain matching authorization, even if the operation is reversible
Reviewed artifact; original grant valid, but a replacement candidate is required	Review	Authorize that replacement; do not transfer the old grant silently
Reviewed artifact; alternative reachable	None	Build a legal continuation channel or stop the path

5. Keep recovery objects available. Rejection must not pollute authoritative state or over-consume grants.
6. Keep audit records atomic with the transition.
7. Separate review, the execution gate, and presentation.
8. Declare the binding rule, the K field set, and the equality relation as design parameters.

The corresponding implementation patterns are:

Instance-bound approval: Persist the reviewed instance and require the submitted instance to match it.

Reuse under grant: Make the reviewed candidate available for a later request under its still-valid authorization.

Reauthorize on a new instance: Obtain an authorization matching the replacement candidate before submitting it.

Handoff guard: Where queued handoffs or re-previews create new identities, check continuity at the execution boundary.

Exposure accounting: Report the opportunity for substitution separately from whether an agent used it.

Audit chain: Retain the candidate, authorization, transition, successor, and source links needed to inspect the relation.

These patterns organize the already specified obligations. They do not guarantee that every application exposes the necessary capabilities or that an agent will select a legal continuation.

A10.5 Status of the design knowledge

The guide is derived from the specification and evaluated mechanisms and has not been validated through a developer study. Its purpose is to help a team implement an explicitly supplied responsibility rule, identify blocked submission paths, avoid silent policy downgrade, and identify where the empirical evidence transfers. It does not estimate production benefit or select the appropriate responsibility rule for that team.

Several candidate hypotheses remain untested: requiring teams to declare a binding rule might reduce implicit substitutions; exposure accounting might change review decisions; and reauthorization channels might reduce abandonment after rejection. These are directions for validation, not additional findings of the present study.

A11 Native approval and binding-scope reproduction

Version and execution scope. The corrected experiment is retained in `research/ase-x1-x2-2026-10-21/v2/` and supplied in `X1-X2-reproduction.zip`. It uses Python 3.11, LangChain 1.2.10, langchain-core 1.2.13, LangGraph 1.0.8 and langgraph-checkpoint 4.0.0. All assistant messages, reviewer decisions and interventions are scripted; no hosted model is called. The protocol is an amendment made after inspection of the initial experiment. The earlier version is retained as historical material; the main text uses the corrected version and its two saved final runs.

X1: pending-interrupt routing and explicit edits. Sixteen conditions exercise the unchanged official LangChain middleware and LangGraph interrupt/resume primitives, using author-written graphs, tools and recorders. Both rounds of each cross-turn test use the same graph, checkpointer and thread. Records separate native behavior, official configuration and adapter choices. Each final run contains 18 actual effects read from SQLite. A new round creates a new pending interrupt: resuming with the old interrupt identifier produces no new effect and leaves the current review pending, whereas the matching identifier allows the current action. Repeating an unkeyed affirmative input at a new interrupt is a fresh input from the caller. Authorized edits retain the earlier proposal and approval payload in checkpoint history. These conditions characterize routing and explicit-edit semantics for the tested interfaces. They do not exercise same-identifier pending-state substitution within a shared interrupted thread, Agent Server authorization policies, or other persistence backends. Matching an application grant to a candidate during queued handoffs or reuse is a separate design question. `RAW-EVIDENCE.json` preserves the framework identifiers, requests and checkpoint history; `OBSERVATIONS.json` records the relationships compared across runs.

X2: binding and execution checks. Nine constructed fixtures cross three main binding comparators and one raw-content diagnostic with two common execution modes: 54 main routes and 18 diagnostic routes per run. The comparators share a request-value guard; the content projection extends the retained context with decision-relevant fields. Amount normalization follows the experiment's declared two-decimal rule. These selected comparators do not implement the manuscript's full G, nor do they replace its canonical-JSON equality. The unchecked executor reads the external document at execution; the verified executor compares its bytes to the reviewed digest and consumes those same bytes. Both modes are applied to every comparator.

For context, normalized content, instance and raw-content binding, respectively, semantic-violation counts are $4, 1, 1, 1$ with unchecked execution and $3, 0, 0, 0$ with verified execution (nine fixtures per cell). Corresponding SQLite-write counts are $8, 5, 2, 4$ and $7, 4, 1, 3$, totaling 34 effects per run. The three remaining context violations concern fields omitted from its retained comparison. The changed-document case is stopped by the shared executor check in every arm. Instance binding also rejects three same-semantics submissions with new identifiers, recording the cost of the selected attribution rule separately from semantic violations. The scorer reads the saved approved-effect fixtures and SQLite in read-only mode without importing the gate or its projection.

Saved runs and verification. The archive's `v2/runs/final-1/` and `v2/runs/final-2/` contain runner metadata, checks, raw evidence, scores and SQLite databases. Their semantic JSON/CSV and actual SQL rows agree. Randomized framework identifiers are retained but not compared byte for byte. The original `VERIFICATION.json` also records historical source-preservation checks at experiment completion; those checks describe that time, before the manuscript was revised. The archive's `verify_saved.py` verifies the preserved artifact hashes, compares the two saved executions and independently rescores the X2 databases without asserting that the later manuscript remains unchanged.

Reproduction entry point. After extracting the archive, `REPRODUCE.md` gives commands for read-only validation and optional fresh execution. The saved evidence can be checked with `python verify_saved.py`; fresh execution uses the pinned requirements and `v2/run_all.py` with new output directories. New runs do not overwrite the archived results. The unaltered `v2/FINAL-MANIFEST.json` pins 111 source and result files and has SHA-256 `7e850e40e45f2afeffed1a2ab5eadb62738343043406049f5a35dc09d55cb066`. These deterministic checks complement the separate paired-agent study; they measure interface and binding behavior rather than autonomous recovery or human-review costs.

A12 Archived recovery availability and matched task routes

This post-hoc readback adds no trajectories to the 32 delivered G pairs. It asks whether the chosen reuse and reauthorization routes were associated with different availability of the reviewed candidate and original grant at the saved rejection boundary. The script `qa/recovery_availability.py` reads the same two frozen archives as the recovery-cost decomposition and checks their whole-file SHA-256 hashes before reading any episode.

For each episode, the script resolves the saved delivered-feedback event and verifies the hashes of its rejection-state and model-input references. It independently checks thirteen predicates: active unconsumed grant, active reviewer, candidate digest, context digest, preservation of the reviewed candidate, predecessor freshness, evidence bytes, evidence locator, source-supported proposal, retained metadata, matching record/field target, matching review context, and visible grant/candidate/authorized value. Here, visible means present in the saved agent input, not assessed human perception or understanding. The script does not use `legal_paths_at_rejection` to decide availability and does not import the admission engine. The rejected attempt preserves the business head and transition list; request and receipt bookkeeping may change. The observed subsequent commit is checked against the hashed before/after snapshots and must add exactly one transition.

All thirteen predicates hold in all 32 episodes. Each visible review response contains the original candidate, authorization identifier and authorized value. The first post-feedback action is a commit in the 24 reuse episodes, or an authorization request followed by a commit in the eight reauthorization episodes. Thus a visible valid reuse option coexists with every observed reauthorization choice. This is an availability statement at a recorded boundary, not a counterfactual replay, an optimality claim or a guarantee against state changes during a production delay.

Table 12: Matched task routes at the 16 G task definitions. Reuse means committing the reviewed candidate under the existing grant; fresh means requesting matching new authorization. All 32 episodes have a visible valid reuse option.

Task suffix	A collection	A route	Original B route
001	Supplementary	Reuse	Reuse
002	Supplementary	Reuse	Reuse
003	Supplementary	Reuse	Reuse
004	Original	Fresh	Reuse
005	Supplementary	Reuse	Reuse
006	Supplementary	Fresh	Reuse
007	Supplementary	Fresh	Reuse
008	Supplementary	Reuse	Reuse
009	Supplementary	Fresh	Reuse
010	Supplementary	Reuse	Reuse
011	Supplementary	Fresh	Reuse
012	Supplementary	Reuse	Reuse
013	Supplementary	Reuse	Reuse
014	Supplementary	Fresh	Reuse
015	Supplementary	Fresh	Reuse
016	Supplementary	Fresh	Reuse

The task-wise comparison is descriptive. A's sixteen episodes combine one original episode with fifteen later episodes selected by the original quota interruption; B's sixteen are original. The same task definition does not make provider, collection period, sampled prefix or hidden serving state exchangeable. No model-effect significance test is inferred from the eight discordant route pairs. The original three cohort ledgers remain unchanged.

Outputs `tables/availability/episodes.csv`, `matched-tasks.csv` and `summary.json` retain every task route, archive member, state/input hash and cohort. The command and the two external archive dependencies are specified in `REPRODUCE-RECOVERY.md`.

Secondary uncertainty analysis of complete suffix cost

The script `qa/scientific_statistics.py` resamples complete task pairs separately within each collection (10,000 draws, seed 20261023). It retains policy pairing and does not pool the A/B observations of a task as independent tasks. The percentile 95% intervals are exploratory secondary analyses of existing records, conditional on a small constructed task set. They are not deployment-level performance guarantees. Original A has one pair, for which no interval is estimated.

The harness performs every context-side handoff commit. The bound continuation must perform a recovery commit after rejection; its full-suffix difference also includes subsequent confirmation and subtracts context-side agent calls. Recovery through the first persistent commit uses the selected route's minimum in all 32 cases, but full-suffix differences do not. Original B's differences are zero in two pairs, one in twelve and two in two; supplementary A's range includes a negative difference. No claim of optimal route selection follows.

Table 13: Complete G suffix cost, bound minus context. Ranges describe observed pairs; intervals are exploratory paired-bootstrap intervals

Collection	Pairs	Mean calls	Call range	95% interval
Original B	16	1.00	$[0, 2]$	$[0.75, 1.25]$
Supplementary A	15	1.60	$[-1, 4]$	$[0.87, 2.33]$
Original A	1	2.00	$[2, 2]$	Not estimated

A13 Exploratory permission-state and feedback comparison

Purpose and provenance. This study was separately frozen before collection on 14 October 2026 (freeze time 13 October, 17:09:12 UTC) and is incorporated into the present revision after reauditing its raw records. It asks whether explicit legal-path labels add value beyond the same visible permission facts, and observes continuation when permission changes after rejection. It is not a new collection for this revision. The four task instances, FORMAL-G-003 through FORMAL-G-006, are the next IDs not used in two preceding feedback minipilots. They reuse the original B review checkpoints from the existing G task family; they are not unseen tasks relative to the full research program. Selection does not use their original suffix outcomes. Earlier minipilot results are not pooled with this comparison.

Design. Each task has three administrative states after the common identity-mismatch rejection: *valid* (old grant and reviewer usable), *revoked* (old grant revoked, reviewer usable), and *unavailable* (reviewer unavailable, no legal path). Candidate content, evidence, target and predecessor remain valid. Two independent arms receive either *facts*, the historical rejection and current permission facts, or *facts plus options*, those identical facts plus derived legal-path labels. Removing the options field leaves the initial visible messages byte-identical; initial states also match in all twelve pairs. The deterministic mapping uses only common visible facts, not hidden goals or a scoring oracle. It is valid for this fixture, not a general concurrent authorization planner.

The twelve task–state pairs run in a seeded order (20261014), with arms adjacent and each state balanced two pairs per starting arm. There are 24 hosted trajectories and 24 separate simple-rule trajectories, one realization per cell. The requested model alias is `deepseek-v4-flash` through the same provider interface as B, at temperature zero. The output cap is 393,216 tokens, the request timeout 1,800 seconds, the episode limit 21,600 seconds and the decision limit six; one transient-transport retry is allowed. The returned alias does not identify an immutable serving revision. These budgets and the collection period differ from the original B study.

Endpoints and complete results. The primary comparison is subsequent tool calls and rejected operations, options minus facts. Feasible completion and unavailable-state stopping have separate denominators. Unavailable episodes have $U = 0$ because the task is not committed; a correct stop requires unchanged business state, no new transition and a truthful report, and is not counted as completion.

Table 14: Exploratory feedback results. Each column contains twelve hosted episodes across four tasks and three permission states. Counts are totals, not independent-task estimates.

Observation	Facts	Facts plus options
Feasible task completion	8/8	8/8
Unavailable-state correct stops	4/4	4/4
Integrity $I = 1$	12/12	12/12
Tool calls	16	16
Commits / authorization requests	8/4	8/4
Rejected continuation operations	0	0
Model requests / responses	28/28	28/28
Elapsed seconds, total	371.048	283.751
Output tokens	66,774	49,293
Provider input tokens	9,747	9,376
Cache-read input tokens	70,400	72,064
Cache-creation input tokens	0	0

The twelve paired tool-call differences are -1 in three, zero in six and $+1$ in three; task-level sums are 0, 0, -1 and $+1$. Facts uses three state queries and one receipt query; options uses four receipt queries. Grouped by permission state, the four paired call differences sum to $+2$ for valid grants, $+1$ for revoked grants and -3 for unavailable review. All three negative differences omit a state query when no legal route

remains; all three positive differences add a receipt query in feasible episodes. These are descriptive contrasts across four reused tasks per state, not estimates of a feedback-by-state interaction. Table 15 retains the corresponding secondary time and output-token totals.

Table 15: Secondary feedback outcomes by permission state. Each row contains four pairs; F is facts and L is facts plus labels

	Calls		Seconds		Output tokens	
State	F	L	F	L	F	L
Valid	5	7	90.407	97.813	15,773	16,863
Revoked	8	9	155.578	142.954	27,799	24,963
Unavailable	3	0	125.063	42.984	23,202	7,467

These legal reads are confirmation behavior, not rejected or invalid actions. The simple-rule controller produces identical action sequences across the two feedback variants, completes all sixteen feasible rule episodes and correctly stops in all eight unavailable rule episodes. It always confirms the final state and therefore uses 24 calls per feedback variant. It is not an optimal controller; the smaller model call total does not establish an LLM-specific advantage.

All 56 hosted responses terminate naturally. Four exceed 4,096 output tokens; the largest has 6,522. No response is truncated, and no retry, transport failure, timeout, quota failure or unfinished episode occurs. Elapsed time includes feedback construction, recording and continuation; it excludes preparation, offline audit and human review. Time decreases in eight of twelve pairs (median paired difference -7.219 seconds), and output tokens decrease in nine. These secondary descriptive directions do not establish a stable overall cost advantage when the primary tool-call outcome has no net improvement.

Options are computed without a model call. Across twelve hosted notices, the options computation takes 37,600 ns in total, and adds 31, 61 or 78 UTF-8 bytes per notice. This in-memory fixture does not measure production database access, concurrent consistency or rule maintenance. Provider usage fields remain separate and are not converted to unverified monetary cost.

Interpretation and reproduction. The comparison supports permission-sensitive continuation under the tested instructions and exposes no net tool-call benefit from path labels. It is not an equivalence test, a prevalence estimate, evidence about real review queues or a cross-model comparison. The four tasks share one family; repeated states do not create twelve independent task samples.

The supplied `Feedback-sensitivity-reproduction.zip` retains the frozen study directory, its pinned source dependencies and a read-only entry point. `qa/feedback_sensitivity.py` verifies eighty frozen files, reconstructs all 48 hosted/rule episodes from saved inputs, responses and state transitions, and recomputes the descriptive analysis. All 48 episode audits pass. The 1,369 files in the study directory remain unchanged by the readback. Source hashes, cell results and all paired cost differences are retained in `tables/feedback-sensitivity/`; the archive's `REPRODUCE.md` specifies a fresh output directory and no provider calls.

The source-package script `qa/feedback_strata.py` groups the saved model pairs by permission state and writes `strata.json`. This grouping adds no trajectories and leaves the frozen experimental archive unchanged.

A14 Native interrupt and relational contract composition

Question and design. This deterministic experiment connects the native interruption mechanism and the relational contract in one scripted execution. It tests whether an application adapter can preserve the reviewed-candidate relation through approval, handoff, rejection and continuation. It uses LangGraph 1.0.8, langgraph-checkpoint 4.0.0, langchain-core 1.2.13 and SQLAlchemy 2.0.23 with Python 3.11. The existing `stores.py`, its source manifest and nineteen contract source files are copied without edits. Native framework code is also unmodified. The protocol, runner, independent scorer, scorer controls and dependencies are frozen as 27 files before two final executions, each with fresh SQLite databases. The development round remains separate.

Adapter responsibilities. An author-written graph first pauses over an already registered immutable candidate proposing `machine proposal` for the address field. The scripted reviewer can deny or authorize `reviewed correction`; only an affirmative response to the current native interrupt creates the application grant handle. A second interrupt is an instrumentation barrier for injecting the controlled handoff, not another approval request. The graph invokes the unchanged relational gate with a complete two-field successor. After rejection, a recovery interrupt receives a scripted choice. Reuse submits the old candidate/grant; fresh

authorization visits a new review interrupt before creating a grant for the replacement. The unchanged batch field supplies a source-preservation check. X3 assumes the trusted, content-immutable candidate storage required in main-text Section 3.2. It tests the resulting identity relations at commit; it does not independently probe same-ID content mutation or object replacement between check and use.

The graph, payload interpretation, grant-handle registry, interventions and recording are adapter code. InMemorySaver and pre-commit grant handles are in process. The database atomically persists the admitted successor and its authorization/transition relations. This design does not provide or test restart-safe approval storage, authenticated review or a distributed transaction between checkpoint and business storage.

Table 16: All ten composition conditions, with identical semantic observations in both final executions. Counts are per execution. "No write" refers to business versions and transactional relations, not candidate registration.

Condition	Observed behavior	Address commits
Corrected approval	Current approval commits corrected value with reviewed identity.	1
Denial	No grant handle or successor write.	0
Identity substitution; reuse	First attempt rejects; old reviewed candidate/grant commits.	1
Identity substitution; fresh review	First attempt rejects; old interrupt ID cannot approve replacement; current ID permits it.	1
Wrong executed value	First attempt rejects; original correction commits on reuse.	1
Stale predecessor	Independent batch update persists; stale address attempt rejects.	0
Rollback after compare-and-swap	No partial transactional rows; retry commits once.	1
Rollback before commit	No partial transactional rows; retry commits once.	1
Duplicate native resume	Completed execution has no additional write.	1
Consumed application-grant replay	Direct replay has no additional write.	1

Independent readback. Every graph invocation and gate attempt retains a SQLite copy and the native pending interrupt identifiers, payloads and scripted input. The scorer imports neither the gate nor LangGraph. It opens fresh read-only SQLite connections and checks version counts, corrected values, the retained batch source, and joins among candidate certificates, authorization bindings, human-decision rows and fact transitions. It also compares the complete record-version, transition, decision and authorization-binding tables before and after every rejected or rolled-back attempt. Gate-return status is not used to determine whether a business effect occurred. Candidate registration is permitted before rejection and is kept separate from transactional business effects.

Both executions satisfy all ten conditions. Each contains fourteen gate attempts: eight admitted corrections and six rejected or rolled-back attempts with no partial transactional rows. The stale case contributes one separate independent batch update and no address correction. Repeating a completed native resume or explicitly replaying a consumed application grant adds no business version. In the fresh-review case, the new native interrupt ID differs from the original; submitting the original ID leaves the current review pending and creates no new grant or write. Submitting the current ID permits the replacement's corrected transition with a matching authorization.

Four negative controls mutate database copies: an incorrect final value, a deleted transition, an incorrect authorized value and an extra version. The independent scorer detects all four. These checks exercise the oracle; they are not additional task successes or security-attack trials.

Interpretation and artifacts. The evidence establishes compatibility of native interrupt routing and the existing relational checks for the specified adapter and scripted paths. It does not establish autonomous recovery, production performance, concurrent replay resistance, restart durability, human review validity or protection against hostile mutation of pending graph state. Two executions test deterministic repeatability on the same machine, not statistical robustness. The agent trajectories reported elsewhere remain a separate evidence layer.

`Native-contract-reproduction.zip` contains `PROTOCOL.md`, `FREEZE.json`, `run.py`, `score.py`, `test_scorer.py`, the unchanged backend, requirements, both final run directories and all SQLite snapshots. `REPRODUCE.md` gives read-only rescoring and fresh-directory rerun commands. No provider access or model calls are needed. Development records are retained separately in the local experiment directory and are not pooled with final observations.

Table 17: Existing foundations and their role in the corrected-execution comparison. P1–P6 are defined in main-text Section 3.2.

Obligation	Existing basis	Role in this study
P1	Continuity Kernel binds proposal and sealed candidate; ReSA tracks the consumed object.	Select context or reviewed-instance agreement as the binding condition while holding correction and shared checks fixed; observe continuation.
P2	ReSA requires preserved or atomically validated consumed versions.	Check internal record, field and evidence consistency against the submitted immutable candidate. Cross-candidate equality belongs to P1; external referents need a separate consumption check.
P3	Guarded Commits binds authority and pre-edit, delta and post-edit hashes.	Check principal authority and the explicit authorized value, retaining the machine proposal separately from the correction and committed value.
P4	Optimistic concurrency control and Continuity Kernel validate predecessors.	Keep freshness shared across policies, separate from instance mismatch.
P5	Transactions and Continuity Kernel install an atomic accepted unit.	Install the complete corrected successor, sources, transitions and grant consumption together.
P6	Provenance and curated-database copying retain derivation and source relations.	Link changed fields to authorized transitions and copy unchanged fields' exact prior sources; check these relations in persistence.

A15 Obligations and existing foundations

The table retains the detailed mechanism-to-obligation mapping. Bibliographic entries for the named mechanisms are provided in the main manuscript's related work. The mapping identifies reused foundations and the role of each obligation in this controlled comparison; it does not assert that existing systems cannot express the same relations.

P1 is policy-relative: context binding requires K_{eq}, and bound binding adds I_{eq}. P2 is context/evidence integrity and P3 is principal/authorized-value integrity in both policies. The source-level Alloy names use a historical numbering scheme; Section A9.5 gives their precise correspondence and verification limits.

The comparison's admission-set inclusions follow from the declared predicates. The store checks assess implementation of the shared obligations; the agent trajectories assess choices and costs after admission or rejection. The observed reuse, renewed authorization and confirmation actions cannot be derived from the binding predicates alone.

A16 Dynamic recovery after an initially available route

This separately frozen exploratory pilot distinguishes route availability at feedback, authorization at execution, and the eventual business effect. It reuses the bound Host and independent oracle without modification, and two preselected checkpoint definitions (FORMAL-G-003 and FORMAL-G-004) from the earlier feedback study. Each initially has a valid reviewed candidate, active grant and available reuse, reauthorization and fresh-proposal routes. The initial visible state is identical across the four conditions of a task. Eight hosted continuations, eight adaptive-rule continuations and eight cached-route controls were executed on 23 October 2026 after freezing the protocol, inputs, code, allocation and limits. They are not added to the original 32 G episodes.

Immediately after the controller chooses its first continuation commit, but before the Host executes it, the harness either leaves state unchanged, revokes the grant named by that action, advances the predecessor, or revokes active grants and disables the reviewer. Selected-grant revocation is an adaptive, controlled intervention, not a sample of naturally occurring administrative changes. After the triggered return, all three controllers receive the same current administrative facts. In the predecessor condition, the existing Host advances the version while preserving the current reference-note value; a fresh candidate and authorization can therefore restore a legitimate path. The unavailable-reviewer condition has no legal continuation route.

All eight hosted first commits were admissible before intervention; only the two static commits remained admissible at execution. The two revocations and two unavailable-reviewer cases returned `AUTHORIZATION_INVALID`; the two predecessor changes returned `STALE_VERSION`. These six deterministic rejections are host safety behavior, not autonomous-recovery successes.

The hosted and adaptive-rule controllers reauthorized after revocation and rebuilt the candidate and authorization after a predecessor advance. All three controllers correctly stopped in both no-legal-route

Table 18: Dynamic pilot: independently read-back business recovery. Each controller has two trajectories per condition; the controllers are not pooled.

Condition	Hosted	Adaptive rule	Cached route
Static	2/2	2/2	2/2
Selected grant revoked	2/2	2/2	0/2
Predecessor advanced	2/2	2/2	0/2
No legal route	0/2	0/2	0/2

cases, with unchanged target values and accurate observed reports. The cached-route controller also stopped after the other dynamic rejections, leaving four feasible corrections incomplete. A fresh SQLite connection independently read the persisted host state: all 24 trajectories had integrity one, and all eight hosted final reports matched that state. Negative controls rejected a forged success report and unrelated-field damage, and returned unknown when independent readback was removed. All 24 saved trajectories were reproduced exactly by offline replay.

A preserved measurement limitation. The frozen scorer additionally defines `task_completion` by requiring a successful `get_state` at the final version, alongside business recovery and an accurate report. This *state-query-confirmed completion* measure is narrower than the original task's allowance for receipt or state evidence. It is 0/6 in feasible hosted continuations: five instead queried a successful receipt, and one used the successful commit response. All six business corrections are independently verified and accurately reported. The frozen zeros are retained, not interpreted as six failed business tasks. Adaptive rules score 6/6 and cached routes 2/6 on this narrower measure. The pilot thus also exposes a measurement dependence on confirmation-tool choice; subsequent studies should measure completion from independent authoritative state and truthful reporting, with the query choice reported separately.

The hosted collection used 35 requests and 27 continuation tool calls; adaptive rules used 28 calls and cached routes 16. All responses returned the alias `deepseek-v4-flash` and ended naturally, with no truncation, invalid output, transport retry or budget exhaustion. Eight decisions, a 32,768-token output cap, 120 seconds per request and 900 seconds per episode were fixed before collection. The raw provider-usage fields and local elapsed times are retained in `Dynamic-recovery-reproduction.zip`; they do not estimate production latency. Two reused task definitions, one realization per cell, controlled interventions and a mutable service alias do not support statistical generalization, a feedback-format comparison or LLM superiority. Matching adaptive-rule outcomes show that the fixture is solvable without an LLM. The evidence establishes a concrete temporal separation between an available route and a subsequently authorized commit, and between gate rejection and actual recovery.

A17 Reviewed-argument delivery across a process boundary

This section reports a separately frozen check of the trusted-adapter premise used in the native-default and native-to-store probes. Those probes assume that a complete reviewed argument object, supplied per pending tool call, is the object the native runtime executes. The check asks whether three pinned SDKs deliver each reviewed object to its intended pending call, and leave the other pending calls unchanged, when the run is paused for review and then continued, including after a clean process exit and a new interpreter resumes the persisted state.

The question is deliberately narrow. The application merges the reviewed patch into a complete five-field argument object and supplies that object for every pending call, including calls the reviewer did not change. The study therefore measures transmission fidelity of the reviewed object, not partial-patch inference or field-level preservation by the SDK. The deterministic model emits the declared calls and then stops; no language model is called, and no provider request is made. A local tool body writes its actual runtime call identifier and actual arguments to SQLite, and a separate SDK-free scorer reads those rows in read-only mode.

The matrix crosses three frameworks, two lifecycles, seven call scenarios, six review edits and two payload families, giving 504 constructed cells. Lifecycles are an in-process continuation and an exit after a native pause whose persisted state is resumed by a newly launched interpreter. Scenarios are one call; two calls with distinct original arguments; and two calls with equal original arguments, each edited in the first, second or both positions. Edits are a text replacement, integer zero, boolean false, empty string, a three-field change and a same-value confirmation. Payload families are ASCII and text with Unicode, quotes and newlines. The protocol, case matrix, adapters, scorer and SDK sources were hashed before collection; a separately

recorded twelve-cell development preflight is excluded from these denominators, and 23 runtime and oracle controls were exercised before the freeze.

Table 19: Reviewed-argument delivery: cells whose persisted effect multiset equals the reviewed expectations exactly. Each lifecycle exercises one native continuation route per framework; the cells are constructed and are not independent samples.

Framework	Lifecycle	Cells	Exact match	Mismatch
Agno 3.1.10	in-process continuation	84	24	60
Agno 3.1.10	exit and fresh-process resume	84	84	0
LangChain 1.4.7 / LangGraph 1.2.16	in-process continuation	84	84	0
LangChain 1.4.7 / LangGraph 1.2.16	exit and fresh-process resume	84	84	0
Pydantic AI 2.60.0	in-process continuation	84	84	0
Pydantic AI 2.60.0	exit and fresh-process resume	84	84	0
Total		504	444	60

All 60 mismatching cells belong to one path, the Agno in-process continuation, and they share one mechanism. In every one of them the first pending call executed with the complete reviewed arguments supplied for the second pending call, while the second call executed its own; the fields that differ follow the edit, so the pattern is not restricted to calls with equal original arguments. Of the 72 multi-call cells on that path, 60 mismatch; the 24 matching cells are the twelve single-call cells and twelve cells whose two reviewed payloads coincide, where sharing cannot change the delivered values. The public issue #8546 already describes shared human-input state across pending calls in this framework, so the matrix reproduces a known mechanism and characterizes its current-version scope. It is not reported as a discovered defect, not confirmed by the maintainers here, and not counted once per cell.

The in-process arm also differs from the fresh arm in its continuation extension point and in the object it continues from. A 54-cell ablation in the same frozen family (Agno only) separates the two. Holding the pause, the reviewed objects and the recorded arguments fixed, the original-run-object route (`continue_run(run_response=...)`) mismatched in 12 of 18 cells (all twelve multi-call cells; the six single-call cells matched), while the run-identifier route (`continue_run(run_id, session_id, requirements)`) matched in all 18 cells in the same process and in all 18 cells after a clean exit and a new interpreter. The mechanism therefore follows the continuation route and the state it continues from, not the process boundary, and the process contrast alone shows no difference. This is a two-contrast decomposition rather than a single-factor isolation, because the entrypoint and object identity change together in the first contrast. The matching paths are equally bounded: LangChain and Pydantic AI use the same documented continuation route in both lifecycles, so their arms differ mainly in process and serialization, and one route per lifecycle is exercised.

The primary observable is the exact multiset of call identifier, tool name and complete typed payload, compared with the reviewed expectations. Field-level diagnostics locate the same differences and carry no additional detection power; the two agreed on every cell. No setup mismatch, unsupported path, collection failure, worker error, native framework error or blocked network attempt occurred. The constructed cells are not an ecosystem sample: three frameworks were chosen for prior support, each lifecycle exercises one route, and no prevalence estimate, confidence interval or significance test applies. What the check supports is conditional and descriptive: on the documented routes of two SDKs and on the Agno fresh-process route, a complete reviewed object reached its intended pending call and the unintended call was unchanged; on the Agno route that continues from the original in-memory run object with more than one pending call, the reviewed values of one call were executed by another, while the run-identifier route was clean in the same process.

Same-identifier substitution. A third frozen check (16 cells, Pydantic AI 2.60.0) applies a controlled injection between the review and the continuation: the state the continuation executes from is replaced while the native call identifier stays unchanged. When the human decision only approves the pending call, the continuation executed the substituted arguments in all four cells that changed the supplied message history, in both lifecycles; substituting the deferred request object alone changed nothing (two of two executed the reviewed object), and the two cells that approve with no substitution also executed the reviewed object. When the decision re-supplies the reviewed object through the native override argument, the reviewed object was executed in all eight override cells: six with substitution and two unchanged controls. These three contrasts are conditional on the tested path: matching native call identifiers did not keep the executed arguments equal to the reviewed ones, while supplying the complete reviewed object through

the native override argument left the executed arguments unchanged under the same substitution. The framework's documentation describes client-supplied message history as input outside the boundary the framework enforces, and does not present native approval as a substitute for server-side authorization (`https://pydantic.dev/docs/ai/core-concepts/message-history/`, accessed 2026-10-24). The result is therefore an empirical statement about an application-maintained responsibility boundary, not a framework defect; post-approval substitution itself is reported in prior work on approval hijacking.

Artifact locations. The three supplied archives contain frozen protocols, code, cases, source seals, raw per-cell databases, scores and reports. Their original freeze SHA-256 values are: `Correction-consumption-reproduction.zip`, `740ccc4fee766849dfaa9fec529c79ba36372cd521138f57672aa586ec89cf8b`; `Agno-entrypoint-reproduction.zip`, `f17ade246c0e92371874b9fc6373461f2f42ca1ff85ec9e0d2c8d95dd8076166`; and `Reviewed-instance-substitution-reproduction.zip`, `b5871cf623af9cb81fa5c17ac034c913345c8b5d6ed0836e29ed14da498d7a66`. In each extracted study directory, `python verify_offline.py` checks package, source and raw-record hashes and recomputes effects from the case definitions using only the standard library. It needs neither SDK imports nor the recorded installation paths. Recollection instead requires pinned SDK environments and configured collectors; their concurrency ladder and no-overwrite/no-replay guards remain unchanged. `REPRODUCE-V4.md` documents these separate workflows and the history intervention. Neither workflow requires model calls.

Artifact sources and reproduction

The following repository-relative directories contain the complete protocols, raw or linked records, independent scoring, and reproduction entry points. `SOURCE-MANIFEST.json` supplied with the manuscript package pins the report versions and archive hashes used for Sections A1–A8 and the current manuscript artifacts. Sections A11 and A14 additionally identify the self-contained X1/X2 and native-to-store archives and their source manifests. Online Resource 1 remains the reference for the original study evidence.

- Output audit: `research/kais-revision-output-audit-2026-10-14/RESULTS.md`.
- Budget sensitivity: `research/kais-budget-sensitivity-2026-10-14/RESULTS.md`; offline analysis uses `analysis.py`.
- Commit uncertainty: `research/kais-commit-uncertainty-2026-10-14/RESULTS.md`; `run_probe.py` with a fresh output directory.
- CK mapping: `research/kais-native-baseline-probe-2026-10-14/RESULTS.md`; `run_probe.py` with a fresh output file.
- OpenTofu integration: `research/kais-approval-workflow-2026-10-14/RESULTS.md`; `run_workflow.py` with a fresh output directory and pinned protocol.
- Historical chain audit: `research/authorization-granularity-phase1-2026-10-06/`; controlled-case records: the pinned DKE-supplement archive cited in the main text.
- N exposure recount: `revisions/2026-10-18-kais-recalibrated/qa/n_exposure.py` and `revisions/2026-10-18-kais-recalibrated/qa/n-exposure.csv`; source archives are named with hashes in Section A7.
- ToolGate development and public-reference comparisons: `research/kais-toolgate-feasibility-2026-10-20/` and `research/kais-toolgate-public-reference-2026-10-20/`; sources, frozen runs and the retained diagnostic are mapped in Section A8.
- Native-approval and binding-scope comparisons: `research/ase-x1-x2-2026-10-21/v2/`; the supplied `X1-X2-reproduction.zip` retains the corrected sources and both final runs, with instructions in `REPRODUCE.md`.
- Reviewed-argument delivery: `research/correction-consumption-study-2026-10-23/`; A17 identifies its archive and two companion checks. Each supplies raw effects and portable offline verification instructions in `REPRODUCE-V4.md`.

Official workflow sources, archived with hashes in the OpenTofu comparison directory, are `https://developer.hashicorp.com/terraform/tutorials/automation/automate-terraform` and `https://opentofu.org/docs/cli/commands/apply/`. The CK paper is already cited in the main manuscript; its pinned model is `https://github.com/openkedge/pci/blob/74841530988f1324bd927da15519486509372917/src/transactional_continuity_kernel/bounded_model.py`.

The reported rebuilds used new directories on the same machine. Independent-environment validation is retained separately: a clean-environment same-machine rebuild (`research/kais-clean-reproduction-2026-10-14/`) and a cross-machine Linux rebuild with Python 3.11.13 (`research/kais-server-reproduction-2026-10-14/`). The post-submission studies in this addendum were not themselves rerun cross-machine.